

Programming Exercise
Control for Spacecraft Rendezvous

Introduction

In this programming exercise, you are in charge of designing a controller for the approaching stage of an orbital rendezvous maneuver, where one satellite approaches another object in space. During the approaching stage, the control system shall steer the chaser satellite into the vicinity of the target. Thereby, the control system should be robust against deviations of the chaser satellite's initial starting position, respect state and input constraints, as well as minimize overall fuel consumption.

Equations of motion

The dynamics of the chaser satellite are described in a moving coordinate frame centered at the target object, which moves on a circular orbit. The resulting second-order ordinary differential equations (ODEs) are the *Clohessy-Wiltshire-Hill (CWH) equations*,

$$\begin{bmatrix} \ddot{x}(t) \\ \ddot{y}(t) \\ \ddot{z}(t) \end{bmatrix} = \begin{bmatrix} 2\omega_n \dot{y}(t) + 3\omega_n^2 x(t) \\ -2\omega_n \dot{x}(t) \\ -\omega_n^2 z(t) \end{bmatrix} + \frac{1}{m} \begin{bmatrix} u_x(t) \\ u_y(t) \\ u_z(t) \end{bmatrix}, \quad (1)$$

where $x(t), y(t), z(t) \in \mathbb{R}$ are the spatial coordinates radially outward, along the orbit track, and along the orbital angular momentum vector of the chaser satellite relative to the target body, respectively, as can be seen in Figure 1. The thrust of the chaser satellite in the relative coordinate frame is given by the control inputs $u_x(t), u_y(t), u_z(t) \in \mathbb{R}$.

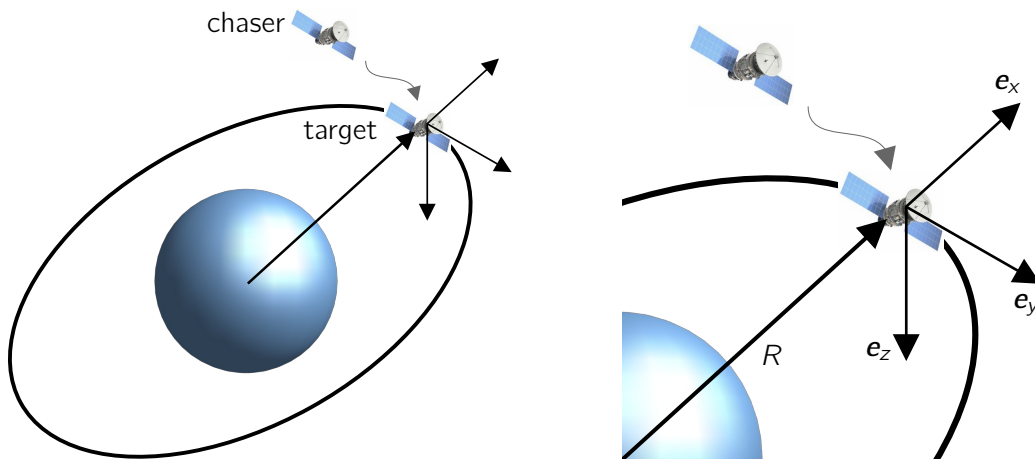


Figure 1: Chaser and target satellite in moving CWH frame.

The model is parametrized by the mass $m := 300\text{kg}$ of the chaser and the *orbital rate* $\omega_n := \sqrt{\frac{\mu}{R^3}}$, where $\mu := 3.986 \cdot 10^{14} \frac{\text{m}^3}{\text{s}^2}$ is the standard gravitational parameter and $R = 7 \cdot 10^6 \text{m}$ is the radius of the target orbit.

Constraints

Let

$$\mathbf{u}(t) := [u_x(t) \ u_y(t) \ u_z(t)]^\top \in \mathbb{R}^{n_u}. \quad (2)$$

The maximum absolute thrust that can be provided by the satellite in each component is limited, i.e.,

$$\|\mathbf{u}(t)\|_\infty := \max\{|u_x(t)|, |u_y(t)|, |u_z(t)|\} \leq u_{\max}, \quad (3)$$

where $u_{\max} := 1\text{N}$. This set of polytopic constraints can be conveniently written as

$$H_u \mathbf{u}(t) \leq \mathbf{h}_u, \quad (4)$$

for some $H_u \in \mathbb{R}^{6 \times n_u}$ and $\mathbf{h}_u \in \mathbb{R}^6$.

Mission objective

The satellite is ejected from the carrier rocket at time $T_0 := 0\text{s}$. We will investigate the approach of the satellite from three different initial conditions which will be defined later. The controller employed for the approaching stage should adhere to the following requirements:

- The fuel consumption should be minimized. As a surrogate for the fuel consumption in the time interval $[0, T]$, we use the $\mathcal{L}_2$ norm; for a piecewise constant input function $\mathbf{u}$, with $\mathbf{u}(\tau) = \mathbf{u}(k\Delta T)$ on the interval $\tau \in [k\Delta T, (k+1)\Delta T)$ for all $k = 0, \dots, N_t - 1$ and $\Delta T \cdot N_t = T$, this results in

$$\|\mathbf{u}\|_{\mathcal{L}_2(0,T)} = \Delta T \left(\sum_{k=0}^{N_t-1} \mathbf{u}(k\Delta T)^\top \mathbf{u}(k\Delta T) \right)^{\frac{1}{2}}.$$

- The satellite should stay within its track limits to avoid collision risk and ensure the validity of the linearized model. The state constraints are given as

$$-s_{\max} \leq x(t) \leq s_{\max} \quad -y_{\max} \leq y(t) \leq y_{\max} \quad -s_{\max} \leq z(t) \leq s_{\max},$$

where $s_{\max} := 10^5\text{m}$ confines the chaser perpendicular to the target's orbital velocity and $y_{\max} := 10^6\text{m}$ confines it along the orbital track. Analogously to the input constraints, the 6 constraints above can be compactly expressed as

$$H_x \mathbf{x}(t) \leq \mathbf{h}_x \quad (5)$$

for some $H_x \in \mathbb{R}^{6 \times n_x}$, $\mathbf{h}_x \in \mathbb{R}^6$ where

$$\mathbf{x}(t) := [x(t) \ y(t) \ z(t) \ \dot{x}(t) \ \dot{y}(t) \ \dot{z}(t)]^\top \in \mathbb{R}^{n_x}. \quad (6)$$

- At final time $T_f := 172800\text{s}$ (2 days), the distance and absolute velocity difference of the chaser should not exceed $d_{f,\max} := 100\text{m}$ and $v_{f,\max} := 1\text{m/s}$, respectively, i.e.,

$$d_f := \sqrt{x(T_f)^2 + y(T_f)^2 + z(T_f)^2} \leq d_{f,\max}, \quad (7)$$

$$v_f := \sqrt{\dot{x}(T_f)^2 + \dot{y}(T_f)^2 + \dot{z}(T_f)^2} \leq v_{f,\max}. \quad (8)$$

Preliminaries

MATLAB

While newer versions of MATLAB may work fine, we recommend to use MATLAB **R2022b** for this project. Since only Matlab 2024b or newer are available in the ETH IT shop, you need to create a Mathworks account with your ETHZ mail address and download MATLAB from the official homepage¹. Furthermore, the Control Systems Toolbox, the Optimization Toolbox, Simulink and **no other toolboxes** ought to be installed to prevent unexpected results.

Installation of MPT & Yalmip

We rely on the Multi-Parametric Toolbox (MPT) for set computations and Yalmip² to setup MPC controllers in Matlab.

To install MPT, complete the following instructions:

1. Go to <https://www.mpt3.org/> and click on "Installation & updating instructions".
2. Download the file `install_mpt3.m`.
3. Run `install_mpt3.m` in MATLAB.

MPT automatically installs Yalmip.

Provided Files

The provided files of this project are structured in three subdirectories: `templates/`, `testing/`, and `utilities/`. Make sure to add the whole provided folder including all files and subdirectories to the **MATLAB path**.

The subdirectory `templates/` contains template files that serve as a basis for your implementation. Do **not** change the input-output structure of the provided functions when implementing your solution. The subdirectory `utilities/` contains readily implemented functions that you can use throughout the project. Below you find a more detailed description of a few important files:

- `generate_params_cwo.m` (Function) — This function returns a struct (called `params` in the following) containing parameters that are shared across most of the functions you implement as part of this project. The fields of the `params` struct and their corresponding parameters are detailed in Table 1.
- `plot_trajectory_cwo.m` (Function) — This function plots the state trajectories and constraints of the satellite model. The inputs are given as state and input trajectories X_t, U_t , the struct array `ctrl_info` as defined in Task 7, as well as `params`.
- `plot_trajectory_z_cwo.m` (Function) — This function plots the state trajectories and constraints of the decoupled z-subsystem of the satellite model. The inputs are as above, with the redefined states and inputs from Task 26.
- You are provided with template files for the deliverables to be implemented by you. Below each task described in the following, you will find a table that summarizes the corresponding deliverables. More information about the functions can be found in the templates and their respective task description.

¹<https://ch.mathworks.com/downloads/>

²<https://yalmip.github.io/>

Field	Value
model.nx	n_x
model.nu	n_u
model.ny	n_y
model.Mass	m
model.GravitationalParameter	μ
model.ScalingMatrix	V
model.TargetRadius	R
model.TimeStep	ΔT
model.A	A
model.B	B
constraints.MaxAbsPositionXZ	$s_{\max}$
constraints.MaxAbsPositionY	$y_{\max}$
constraints.MaxAbsThrust	$u_{\max}$
constraints.MaxFinalPosDiff	$d_{f,\max}$
constraints.MaxFinalVelDiff	$v_{f,\max}$
constraints.InputMatrix	H_u
constraints.InputRHS	h_u
constraints.StateMatrix	H_x
constraints.StateRHS	h_x
exercise.SimHorizon	N_t
exercise.InitialConditionA	x_0^A
exercise.InitialConditionB	x_0^B
exercise.InitialConditionC	x_0^C
exercise.QdiagOptA	Q^*

Table 1: params struct of system parameters returned by the function `generate_params_cwo`.

The folder `testing/` contains a number of encrypted MATLAB p-files to test your solution, as well as the `run_tests` function. Usage of the testing framework is described in more detail in the following.

Testing framework

In this programming exercise we make use of an automated testing framework. It serves to provide you with feedback about your solution during development. To facilitate automatic testing of your implementation, most deliverables are given in terms of *functions*, whose input-output behavior is specified in the task description and verified by the testing framework.

To run the tests for all functions, run

```
test_struct = run_tests();
```

The testing framework is available for all tasks where you implement functions. We do not provide testing for tasks where you have to write a script, i.e., Tasks 11, 23 and 31. To run the tests for one specific function called "function_name" ("function_name" is a string, e.g., "generate_system_cont"), run

```
test_struct = run_tests("function_name");
```

The output `test_struct` is a MATLAB struct whose fields are described in Table 2. Note that the tests first check that the tests for any prerequisite functions pass. The testing framework may vary **all** of the inputs of the function to be tested, so it is important to **avoid hard-coding parameters**

such as input and state dimensions etc. As an emphasizing remark, always use the parameters available in the `params` struct in your implementation and do not recompute or hard-code them.

Field	Value
Deliverable	The name of the tested function.
Info	Short summary of the diagnosis. Passed tests are labeled "OK". If an error is caught during the execution of your function, it will display "ERROR: <message>". If the output of your function does not match the expected value, it shows "Failure: <message>".
TestResults	An array of MATLAB <code>TestResult</code> objects, which you can explore for debugging.

Table 2: `test_struct` struct of test results returned by the function `run_tests`.

Scope of the programming exercise

This project is ungraded and should serve as a learning experience for you, hence you do not have to submit your solutions. We highly encourage you to solve the programming exercise to gain practical experience and gain an implementation-level understanding of the concepts learned in the lecture. From experience, many concepts are much better understood and learned when implemented and tested in practice. We also strongly encourage you to collaborate in small groups to solve and discuss the tasks. A group size of up to three students has been ideal in the past. A dedicated recitation will be held towards the end of the semester in the regular recitation slot, where we will discuss the project solution.

Should you have questions while solving the project, please ask them in the dedicated forum section "Programming Exercise - Questions" on Moodle. We will try to answer your questions as soon as possible. In case we (or you) find a bug inside the software framework, it might be necessary to update the testing framework while you are working on the project. In this regard, we highly appreciate your feedback and would like to encourage you to report bugs or unexpected behavior of the software in the same Moodle forum.

Simulation and self-study questions

Some of the questions are marked as simulation or self-study questions. These types of questions are intended to guide your learning experience and foster your broader understanding beyond the pure implementation of the methods. Simulation questions encourage you to test the developed methods in a setting where the differences between different control methods become apparent. Self-study questions are of a more theoretic and experimental nature; they serve as additional material to prepare you for the exam. We thus strongly recommend to answer and discuss these questions among colleagues.

Tasks

In the following, you find the tasks and corresponding deliverables for this project.

System Modeling

Tasks

1. Rewrite the second-order ODE (1) as a first-order ODE of the form

$$\dot{\mathbf{x}}(t) = A_c \mathbf{x}(t) + B_c \mathbf{u}(t),$$

where $\mathbf{x}(t)$ and $\mathbf{u}(t)$ are defined as in equations (2) and (6). Implement a function called `generate_system_cont_cwo` that takes the `params` struct as input and returns A_c, B_c .

2. Discretize the continuous-time ODE using an exact discretization with a sampling time of $\Delta T = 600\text{s}$ such that the resulting discrete-time dynamics are given by the difference equation

$$\tilde{\mathbf{x}}_d(k+1) = \tilde{A} \tilde{\mathbf{x}}_d(k) + \tilde{B} \mathbf{u}_d(k), \quad (9)$$

where $\tilde{\mathbf{x}}_d(k) := \tilde{\mathbf{x}}(\Delta T \cdot k)$ and $\mathbf{u}_d(k) := \mathbf{u}(\Delta T \cdot k)$. Implement your solution as a function `discretize_system`, that computes $\tilde{A}, \tilde{B}$ based on the continuous-time matrices A_c, B_c and the `params` struct as inputs.

Hint: You can use the MATLAB function `c2d`.

3. For the problem to be numerically well-conditioned, the states are re-scaled with the following transformation

$$\mathbf{x}_d(k) = V \tilde{\mathbf{x}}_d(k) \quad (10)$$

where $V = \text{diag}([10^{-6}, 10^{-6}, 10^{-6}, 10^{-3}, 10^{-3}, 10^{-3}]) = \text{params.model.ScalingMatrix}$ is a diagonal scaling matrix. Implement a function `generate_system_scaled`, which takes as inputs the matrices $\tilde{A}, \tilde{B}$, `params` and outputs A, B such that

$$\mathbf{x}_d(k+1) = A \mathbf{x}_d(k) + B \mathbf{u}_d(k) \quad (11)$$

describes the same dynamics as equation (9). Note that this transformation amounts to changing the units of the state from $[\text{m}; \text{m/s}]$ to $[\text{Mm}; \text{km/s}]$. All further exercises are expressed in this new transformed units. For brevity, and with a slight abuse of notation, we drop the subscript in the following and write $\mathbf{x}(k)$ and $\mathbf{u}(k)$ to refer to the discrete state and input variables, respectively.

4. Find matrices $H_u, H_x \in \mathbb{R}^{6 \times n_x}$, as well as vectors $\mathbf{h}_u, \mathbf{h}_x \in \mathbb{R}^6$ to express the state and input constraints in the form of equations (4) and (5), respectively (in the transformed coordinates corresponding to equation (10)). Implement a function `generate_constraints_cwo`, that takes as input the parameter struct `params` and outputs $H_u, \mathbf{h}_u, H_x, \mathbf{h}_x$.
5. Modify the function `generate_params_cwo` to also compute $A, B, H_u, \mathbf{h}_u, H_x, \mathbf{h}_x$ as part of its execution (using the functions you implemented above) and add the corresponding fields as defined in Table 1 to the output struct. The output of the function `generate_params` should now exactly give a struct as specified in Table 1.

Task	Function	Inputs	Outputs
1	generate_system_cont_cwo	params	A_c, B_c
2	discretize_system	A_c, B_c, params	$\tilde{A}, \tilde{B}$
3	generate_system_scaled	$\tilde{A}, \tilde{B}, \text{params}$	A, B
4	generate_constraints_cwo	params	$H_u, \mathbf{h}_u, H_x, \mathbf{h}_x$
5	generate_params_cwo (modify)		params

Table 3: Deliverable summary "System modeling"

Unconstrained Optimal Control

The aim of the following tasks is to design a discrete-time infinite-horizon linear quadratic regulator (LQR) that satisfies the mission requirements. The infinite-horizon LQR controller

$$\mathbf{u}(k) := F_\infty \mathbf{x}(k) \quad (12)$$

is defined such that it minimizes the infinite-horizon quadratic cost

$$J_\infty(\mathbf{x}(0)) := \sum_{k=0}^{\infty} \mathbf{x}(k)^\top Q \mathbf{x}(k) + \mathbf{u}(k)^\top R \mathbf{u}(k), \quad (13)$$

for some positive definite weighting matrices $Q \in \mathbb{R}^{n_x \times n_x}$ and $R \in \mathbb{R}^{n_u \times n_u}$. As the mission requirements and constraints cannot be encoded directly, but rather have to follow from the choice of the weights, the main difficulty is to find Q and R that lead to a satisfactory system response. Therefore a parameter study is to be conducted. To simplify the parameter study, we define $\mathbf{q} \in \mathbb{R}^{n_x \times 1}$, $\mathbf{q} := [q_x \quad q_y \quad q_z \quad q_{vx} \quad q_{vy} \quad q_{vz}]^\top$ and choose³ the parametrization $Q := \text{diag}(\mathbf{q})$ and $R = I_{n_u}$. The parameter study is split into a set of subproblems that form the deliverables for this task.

Tasks

- Design an infinite-horizon LQR controller for given inputs Q and R . Implement your solution as a *class* in the template file `LQR.m`. Note that the class template has two functions, the constructor `LQR(Q,R)` and `eval(x)`. For numerical efficiency, the feedback matrix F_∞ should only be computed at initialization once (in the constructor of the class) and stored as a class property `K := L_\infty`. The class property can then be accessed at every call to the `eval` function, which computes the feedback of the controller according to equation (12) without additional computational overhead. Note that, in addition to the control action $\mathbf{u}$, the `eval` function also returns a struct `ctrl_info` containing additional information about the control output. For the LQR controller, the `ctrl_info` struct has only one field, `ctrl_feas`, indicating the feasibility of the control problem; it suffices to always set

$$\text{ctrl_info.ctrl_feas} = \text{true}.$$

For convenience, we have already added the `eval` function to the template `LQR.m`. More details can be found in the template file.

- Simulate the closed-loop system for a given initial condition $\mathbf{x}(0)$, controller object `ctrl` and number of time steps N_t . Implement your solution in the function `simulate`, which

³Note that the selection of $R = I_{n_u}$ is not restrictive since scaling the infinite-horizon cost does not change its minimizer. The restriction here is mainly introduced by assuming diagonal weight terms, i.e., no coupling of the different states or inputs in the cost.

takes $\mathbf{x}(0)$, `ctrl`, `params` as inputs and outputs the closed-loop trajectory according to equation (11) in terms of the matrices $X_t \in \mathbb{R}^{n_x \times (N_t+1)}$, $U_t \in \mathbb{R}^{n_u \times N_t}$, with

$$X_t := [\mathbf{x}(0) \quad \dots \quad \mathbf{x}(N_t)], \quad U_t := [\mathbf{u}(0) \quad \dots \quad \mathbf{u}(N_t-1)], \quad (14)$$

as well as an N_t -dimensional array of the `ctrl_info` structs described in Task 6. Note that the simulation should explicitly **not** include saturation of the inputs, i.e., compute the evolution of the linear system according to the raw input of the feedback controller.

8. [Simulation]: Experiment with different values of the parameters in $\mathbf{q}$. What is their effect on the resulting closed-loop trajectory with initial condition $\mathbf{x}_0^A$ (see Table 1)? You can use the provided function `plot_trajectory_cwo` to visualize your results.
9. Check the satisfaction of the constraints for a given trajectory. Implement a function called `traj_constraints_cwo`, that takes as input the trajectory data X_t, U_t and computes as outputs
 - the maximum absolute value of both $x(k)$ and $z(k)$, $s_{\max}^{(i)} := \max_{k \in [0, N_t]} \max\{|x(k)|, |z(k)|\}$,
 - the maximum absolute value of y , $y_{\max}^{(i)} := \max_{k \in [0, N_t]} |y(k)|$,
 - the maximum absolute value of the applied thrust, $u_{\max}^{(i)} := \max_{k \in [0, N_t-1]} \|\mathbf{u}(k)\|_{\infty}$,
 - the closed-loop finite horizon input cost, $J_u^{(i)} := \sum_{k=0}^{N_t-1} \mathbf{u}(k)^\top \mathbf{u}(k)$,
 - the distance from the target position at T_f , see equation (7),
 - the absolute difference from the target velocity, see equation (8),
 - a boolean flag `traj_feas(i)` indicating the feasibility of the trajectory, i.e., `traj_feas(i) = true` if and only if

$$s_{\max}^{(i)} \leq s_{\max}, \quad y_{\max}^{(i)} \leq y_{\max}, \quad u_{\max}^{(i)} \leq u_{\max}, \quad d_f^{(i)} \leq d_{f,\max}, \quad v_f^{(i)} \leq v_{f,\max}. \quad (15)$$

10. Perform a parameter study to find a good choice for $\mathbf{q}$. Implement a function `lqr_tuning_cwo`, which takes as input an initial state $\mathbf{x}(0) \in \mathbb{R}^{n_x}$, as well as an array of parameter vectors $\mathbf{Q} \in \mathbb{R}^{n_x \times M}$, $\mathbf{Q} := [\mathbf{q}^{(1)} \quad \dots \quad \mathbf{q}^{(M)}]$, and outputs an M -dimensional array of structs `tuning_struct`. Each element `tuning_struct(i)` should contain the following fields:

Field	Value
<code>InitialCondition</code>	$\mathbf{x}_0$
<code>Qdiag</code>	$\mathbf{q}^{(i)}$
<code>MaxAbsPositionXZ</code>	$s_{\max}^{(i)}$
<code>MaxAbsPositionY</code>	$y_{\max}^{(i)}$
<code>MaxAbsThrust</code>	$u_{\max}^{(i)}$
<code>InputCost</code>	$J_u^{(i)}$
<code>MaxFinalPosDiff</code>	$d_f^{(i)}$
<code>MaxFinalVelDiff</code>	$v_f^{(i)}$
<code>TrajFeasible</code>	<code>traj_feas⁽ⁱ⁾</code>

Table 4: Fields of `tuning_struct(i)`

The second output argument of the function shall be the index $i_{\text{opt}} \in \{1, \dots, M\}$, which corresponds to the index of the LQR controller that is feasible and requires the lowest amount of fuel, i.e., $i_{\text{opt}} := \arg \min_i \left\{ J_u^{(i)} \mid \text{traj_feas}^{(i)} = \text{true} \right\}$. If there exists no feasible LQR controller, the function should set $i_{\text{opt}} := \text{nan}$.

11. Use the function `lqr_tuning_cwo` you implemented above to identify a parameter vector $\mathbf{q}$, such that the corresponding LQR controller is feasible and has a low input cost $J_u \leq 8$ for the initial condition $\mathbf{x}_0^A$. Provide a script file `lqr_tuning_script_cwo.m` which performs the parameter study and finally sets the parameter $\mathbf{q} := \mathbf{q}$ corresponding to the best controller parametrization. Save the variable $\mathbf{q}$, as well as the `tuning_struct` of your parameter study in the MAT-file `lqr_tuning_script_cwo.mat`. Use the naming convention specified in Table 6.

Hint 1: Consider searching in the vicinity of the parameters we provide in the next section, and using the MATLAB function `ndgrid` to get efficient parameter samples for the input matrix $\mathbf{Q}$. You might have to perform several rounds of tuning and refine your sampling grid iteratively to arrive at a satisfactory solution.

Hint 2: Note that the "in-plane" dynamics of the x, y -coordinates are decoupled from the "out-of-plane" dynamics of the z -coordinate. This means you can perform the tuning with respect to the parameters q_x, q_y, q_{vx}, q_{vy} separately from the tuning for the parameters q_z, q_{vz} .

12. [Simulation]: Simulate the closed-loop system with the LQR controller obtained in Deliverable 11, starting from the initial condition $\mathbf{x}_0^B$. Is your controller still feasible?

Task	Function	Inputs	Outputs
6	LQR	$\mathbf{Q}, \mathbf{R}$, params	ctrl (LQR object)
6	LQR/eval	$\mathbf{x}$	$\mathbf{u}$, ctrl_info
7	simulate	$\mathbf{x}_0$, ctrl, params	$\mathbf{X}_t, \mathbf{U}_t$, ctrl_info
9	traj_constraints_cwo	$\mathbf{X}_t, \mathbf{U}_t$, params	$s_{\max}^{(i)}, y_{\max}^{(i)}, u_{\max}^{(i)}, J_u^{(i)}, d_f^{(i)}, v_f^{(i)}, \text{traj_feas}^{(i)}$
10	lqr_tuning_cwo	$\mathbf{x}_0, \mathbf{Q}$, params	tuning_struct, i_{opt}
11	lqr_tuning_scrip_cwo	-	lqr_tuning_script_cwo.mat

Table 5: Deliverable summary "Unconstrained Optimal Control"

Name	Variable
$\mathbf{q}$	Parameter $\mathbf{q}$ corresponding to the best controller parametrization
tuning_struct	Array of structs of your parameter study (output of the <code>lqr_tuning</code> function)

Table 6: Naming convention for `lqr_tuning_script_cwo.m` and `lqr_tuning_script_cwo.mat`

From LQR to MPC

After designing the unconstrained optimal controller, in this section you will design a first simple model predictive controller. In MPC, the control sequence $U := \{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}\}$ is computed as the solution of an optimization problem over a prediction horizon of N steps. Feedback is introduced by only applying the first element of the sequence, $\mathbf{u}(0) := \mathbf{u}_0$; in the next time step, the optimal input sequence is recomputed based on updated state measurements. Note that we write predicted control inputs with a lower subscript, e.g., $\mathbf{u}_0$, to differentiate them from the implemented control inputs $\mathbf{u}(0)$. The notation of the states is chosen analogously.

For the following exercises, we use $Q^* := \text{diag}(\mathbf{q}^*)$ and $R^* := I_{n_u}$, with

$$\mathbf{q}^* := \begin{bmatrix} q_x^* \\ q_y^* \\ q_z^* \\ q_{vx}^* \\ q_{vy}^* \\ q_{vz}^* \end{bmatrix} := \begin{bmatrix} 94.0 \\ 0.1579 \\ 300 \\ 0.01 \\ 0.10 \\ 0.10 \end{bmatrix}.$$

Nevertheless, feel free to also experiment with your best values obtained from the LQR controller tuning and see how it affects the MPC closed-loop performance.

Tasks

13. Explicitly compute the maximum positively invariant set $\mathcal{X}_{LQR}$ under application of the LQR controller, i.e., the set of initial conditions for which the closed-loop system under the LQR controller satisfies state and input constraints for all times. Let $\mathbf{x}_{LQR}(k)$ and $\mathbf{u}_{LQR}(k)$, $k = 0, \dots, \infty$ be the infinite-horizon closed-loop state and input sequence resulting from application of the LQR controller (12) to system (11), for some initial condition $\mathbf{x}_{LQR}(0) = \mathbf{x}(0)$. Then, the set $\mathcal{X}_{LQR}$ is defined by

$$\mathcal{X}_{LQR} := \{\mathbf{x} \mid \mathbf{x}_{LQR}(0) = \mathbf{x}, H_x \mathbf{x}_{LQR}(k) \leq \mathbf{h}_x, H_u \mathbf{u}_{LQR}(k) \leq \mathbf{h}_u \text{ for all } k \geq 0\}. \quad (16a)$$

Implement a function `lqr_maxPI`, which takes as inputs Q, R, params and computes $\mathcal{X}_{LQR}$ in terms of polytopic constraints, i.e., returns $H \in \mathbb{R}^{n_H \times n_x}$ and $\mathbf{h} \in \mathbb{R}^{n_H}$ such that $\mathcal{X}_{LQR} = \{\mathbf{x} \mid H\mathbf{x} \leq \mathbf{h}\}$.

Hint: You can use the MPT toolbox for this task.

14. [Self-study]: Check whether $\mathbf{x}_0^A$, $\mathbf{x}_0^B$, and $\mathbf{x}_0^C$ are contained in $\mathcal{X}_{LQR}$. What can you conclude from the result with respect to state and input constraint satisfaction under the LQR controller for these initial conditions?
15. Implement a function `traj_cost` that takes as input arguments $Q \in \mathbb{R}^{n_x \times n_x}$, $R \in \mathbb{R}^{n_u \times n_u}$, as well as trajectory data X_t, U_t as in equation (14) and outputs the closed-loop quadratic cost for a given trajectory, i.e.,

$$J_{N_t} := \sum_{k=0}^{N_t-1} \mathbf{x}(k)^\top Q \mathbf{x}(k) + \mathbf{u}(k)^\top R \mathbf{u}(k)$$

16. Implement a model predictive controller that solves the open-loop optimization problem

$$\min_U \sum_{i=0}^{N-1} \mathbf{x}_i^\top Q \mathbf{x}_i + \mathbf{u}_i^\top R \mathbf{u}_i + l_f(\mathbf{x}_N) \quad (17a)$$

$$\text{s.t. } \mathbf{x}_0 = \mathbf{x}(k) \quad (17b)$$

$$\mathbf{x}_{i+1} = A\mathbf{x}_i + B\mathbf{u}_i, \quad i = 0, \dots, N-1 \quad (17c)$$

$$H_x \mathbf{x}_i \leq \mathbf{h}_x, \quad i = 0, \dots, N \quad (17d)$$

$$H_u \mathbf{u}_i \leq \mathbf{h}_u, \quad i = 0, \dots, N-1 \quad (17e)$$

at each time step, where $l_f(\mathbf{x}) = J_\infty(\mathbf{x})$ is equal to the LQR infinite-horizon cost (13). As for the LQR controller, implement the model predictive controller as a class MPC, which creates a YALMIP solver object during initialization with Q, R, N , and solves the optimization problem in the `eval` method. Note that in this case, at each call of `eval` the `ctrl_info` struct has two additional fields: `ctrl_info.objective`, which should contain the value of the objective function for the optimizer U^* of (17), and `ctrl_info.solvetime`, which should contain the time required to solve the problem (17). Again, we have provided you with the full `eval` method. More details can be found in the template file.

Hint 1: The LQR infinite-horizon cost is of the form $J_\infty(\mathbf{x}) = \mathbf{x}^\top P \mathbf{x}$, where $P \in \mathbb{R}^{n_x \times n_x}$ is a constant matrix that can be computed during the initialization of the controller.

Hint 2: You can use the MATLAB functions `tic` and `toc` to get an estimate of the required solve time.

17. [Simulation]: Simulate the closed-loop system starting from $\mathbf{x}_0^A$ with the LQR and the model predictive controller for the same choice of $Q := Q^*, R := R^*$ and with $N := 30$. Do the same with $\mathbf{x}_0^B$ as initial condition. How do the controllers perform with respect to closed-loop constraint satisfaction and closed-loop cost?

Task	Function	Inputs	Outputs
13	<code>lqr_maxPI</code>	Q, R, params	H, h
15	<code>traj_cost</code>	X_t, U_t, Q, R	J_N
16	<code>MPC</code>	Q, R, N, params	<code>ctrl</code> (MPC object)
16	<code>MPC/eval</code>	$\mathbf{x}$	$\mathbf{u}, \text{ctrl_info}$

Table 7: Deliverable summary "From LQR to MPC"

MPC with theoretical closed-loop guarantees

In this part, the task is to formulate a model predictive controller that provides guaranteed closed-loop state and input constraint satisfaction and renders the origin an asymptotically stable equilibrium point of the closed-loop system.

Tasks

18. Implement a model predictive controller based on the MPC problem

$$\min_{\mathbf{U}} \sum_{i=0}^{N-1} \mathbf{x}_i^\top \mathbf{Q} \mathbf{x}_i + \mathbf{u}_i^\top \mathbf{R} \mathbf{u}_i \quad (18a)$$

$$\text{s.t. } \mathbf{x}_0 = \mathbf{x}(k) \quad (18b)$$

$$\mathbf{x}_{i+1} = \mathbf{A} \mathbf{x}_i + \mathbf{B} \mathbf{u}_i, \quad i = 0, \dots, N-1 \quad (18c)$$

$$\mathbf{H}_x \mathbf{x}_i \leq \mathbf{h}_x, \quad i = 0, \dots, N \quad (18d)$$

$$\mathbf{H}_u \mathbf{u}_i \leq \mathbf{h}_u, \quad i = 0, \dots, N-1 \quad (18e)$$

$$\mathbf{x}_N = \mathbf{0}, \quad (18f)$$

in the class template MPC_TE, with the same input arguments as the MPC class.

19. [Self-study]: Why is the origin an asymptotically stable equilibrium point for the resulting closed-loop system, given that (18) is feasible for $\mathbf{x}(0)$?

20. Implement another model predictive controller based on the MPC problem

$$\min_{\mathbf{U}} \sum_{i=0}^{N-1} \mathbf{x}_i^\top \mathbf{Q} \mathbf{x}_i + \mathbf{u}_i^\top \mathbf{R} \mathbf{u}_i + l_f(\mathbf{x}_N) \quad (19a)$$

$$\text{s.t. } \mathbf{x}_0 = \mathbf{x}(k) \quad (19b)$$

$$\mathbf{x}_{i+1} = \mathbf{A} \mathbf{x}_i + \mathbf{B} \mathbf{u}_i, \quad i = 0, \dots, N-1 \quad (19c)$$

$$\mathbf{H}_x \mathbf{x}_i \leq \mathbf{h}_x, \quad i = 0, \dots, N \quad (19d)$$

$$\mathbf{H}_u \mathbf{u}_i \leq \mathbf{h}_u, \quad i = 0, \dots, N-1 \quad (19e)$$

$$\mathbf{x}_N \in \mathcal{X}_{LQR}, \quad (19f)$$

in the class template MPC_TS, where $l_f(\mathbf{x}) := J_\infty(\mathbf{x})$ is the LQR infinite-horizon cost and $\mathcal{X}_{LQR}$ from equation (16) is given in terms of $\mathbf{H}$ and $\mathbf{h}$ as outlined in Deliverable 13. Note that the class is initialized with the additional input arguments $\mathbf{H}$ and $\mathbf{h}$, hence $\mathcal{X}_{LQR}$ has to be computed outside of the class.

21. [Simulation]: Simulate the closed-loop system with the three MPCs (17), (18), (19) starting from $\mathbf{x}_0^A$ for the same choice of $\mathbf{Q} := \mathbf{Q}^*$, $\mathbf{R} := \mathbf{R}^*$ and horizon length $N = 30$ and compare them in terms of the feasibility of the open-loop optimization problems, as well as their constraint satisfaction and cost in closed-loop. Test also different horizon lengths $N = 40 \dots 20$. What do you observe?

Task	Function	Inputs	Outputs
18	MPC_TE	Q, R, N , params	ctrl (MPC_TE object)
18	MPC_TE/eval	$\mathbf{x}$	$\mathbf{u}$, ctrl_info
20	MPC_TS	$Q, R, N, H, \mathbf{h}$, params	ctrl (MPC_TS object)
20	MPC_TS/eval	$\mathbf{x}$	$\mathbf{u}$, ctrl_info

Table 8: Deliverable summary "MPC with theoretical closed-loop guarantees"

Soft constraints

In practical implementations, model predictive controllers such as (19) can become infeasible despite the provided theoretical guarantees, for instance due to unmodeled disturbances or model mismatch. This problem can be addressed by using soft constraints, providing a recovery mechanism given that the original problem is infeasible. Your task is to design a soft-constrained model predictive controller, which provides the same control inputs as (19), if (19) is feasible, but may provide a feasible solution even when (19) is infeasible.

Tasks

22. Introduce slack variables $\epsilon_i \in \mathbb{R}^6$, $i = 0, \dots, N$, into the MPC problem (19) to restore feasibility in the case of state constraint violations. Implement a model predictive controller based on the MPC problem

$$\min_{\mathbf{u}} \sum_{i=0}^{N-1} \mathbf{x}_i^\top Q \mathbf{x}_i + \mathbf{u}_i^\top R \mathbf{u}_i + l_f(\mathbf{x}_N) + \sum_{i=0}^N \epsilon_i^\top S \epsilon_i + v \|\epsilon_i\|_\infty \quad (20a)$$

$$\text{s.t. } \mathbf{x}_0 = \mathbf{x}(k) \quad (20b)$$

$$\mathbf{x}_{i+1} = A\mathbf{x}_i + B\mathbf{u}_i, \quad i = 0, \dots, N-1 \quad (20c)$$

$$H_x \mathbf{x}_i \leq \mathbf{h}_x + \epsilon_i, \quad i = 0, \dots, N \quad (20d)$$

$$H_u \mathbf{u}_i \leq \mathbf{h}_u, \quad i = 0, \dots, N-1 \quad (20e)$$

$$\epsilon_i \geq 0, \quad i = 0, \dots, N \quad (20f)$$

$$\mathbf{x}_N \in \mathcal{X}_{LQR} \quad (20g)$$

in the class MPC_TS_SC. The class shall be initialized with the same parameters as MPC_TS, plus the additional values for $S \in \mathbb{R}^{6 \times 6}$ and $v \in \mathbb{R}$ for the constraint violation penalty with $v \gg 0$.

23. Choose S and v such that the controller based on the soft-constrained MPC problem (20) returns the same control input values as the controller based on the MPC problem (19), whenever (19) is feasible, for the same choice of weighting matrices Q^* , R^* and horizon length $N = 30$. Verify your selection in simulation by writing a script called MPC_TS_SC_script_cwo.m. Provide the script and save your selected values in the MAT-file MPC_TS_SC_params_cwo.mat. Use the naming convention specified in Table 10.
24. [Simulation]: Repeat the simulation for the initial condition $\mathbf{x}_0^C$ and experiment with different values for S and v in the soft-constrained formulation (20). How does the choice of S and v influence the closed-loop behavior?

Task	Function	Inputs	Outputs
22	MPC_TS_SC	$Q, R, N, H, \mathbf{h}, S, v, \text{params}$	ctrl (MPC_TS_SC object)
22	MPC_TS/eval	$\mathbf{x}$	$\mathbf{u}$, ctrl_info
23	MPC_TS_SC_script_cwo	-	MPC_TS_SC_params_cwo.mat

Table 9: Deliverable summary "Soft constraints"

Name	Variable
S	Selected value for variable S
v	Selected value for variable v

Table 10: Naming convention for MPC_TS_SC_script_cwo.m and MPC_TS_SC_params_cwo.mat

Robust MPC

If we want to ensure satisfaction of constraints even under uncertain model descriptions, such as model mismatch or unmodeled disturbances, we can use a robust MPC approach. In the given robust MPC task, we just consider the (decoupled) subsystem in z-direction of the overall system (11) after applying the transform (10). Consequently, we consider the following uncertain system

$$\mathbf{x}_z(k+1) = A_z \mathbf{x}_z(k) + B_z \mathbf{u}_z(k) + \mathbf{w}(k), \quad (21)$$

where $\mathbf{x}_z(k) = [z(k) \ v_z(k)]^\top \in \mathbb{R}^2$, $\mathbf{u}_z(k) \in \mathbb{R}$, and the disturbance $\mathbf{w}(k) \in \mathbb{R}^2$ models the uncertainty in the system. The disturbances are constrained to $H_w \mathbf{w}(k) \leq \mathbf{h}_w$ for all time steps k with

$$H_w := \begin{bmatrix} 1 & 0 \\ -1 & 0 \\ 0 & 1 \\ 0 & -1 \end{bmatrix}, \quad \mathbf{h}_w := w_{\max} \cdot \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \quad (22)$$

where $w_{\max} := 10^{-4}$.

We provided you with the function `generate_params_z_cwo` which takes as an input your `params` struct and provides the struct `params_z`, in which $A, B, n_x, n_u, H_x, \mathbf{h}_x, H_u$, and $\mathbf{h}_u$ are adapted to represent the uncertain subsystem in z-direction (21). To keep notation simple and for your reference in the implementation, we redefine $n_x := 2, n_u := 1$; for the updated values of the other variables, please refer to the provided function `generate_params_z`. Note that the redefined variables should not affect your implementation, which should work for either set of variables (as long as the dimensions match). The initial conditions $\mathbf{x}_0^A, \mathbf{x}_0^B$, and $\mathbf{x}_0^C$ are removed, and the initial condition $\mathbf{x}_{z,0}^A$ is added. Additionally, the function `generate_params_z` adds the fields shown in Table 11 to the `params_z` struct to represent the constraints on the disturbances (22).

Field	Value
<code>model.InitialConditionA_z</code>	$\mathbf{x}_{z,0}^A$
<code>constraints.DisturbanceMatrix</code>	H_w
<code>constraints.DisturbanceRHS</code>	$\mathbf{h}_w$
<code>constraints.MaxAbsDisturbance</code>	$w_{\max}$

Table 11: Fields added to `params_z` struct by the function `generate_params_z`.

Tasks

25. Sample a sequence W_t of N_t disturbance vectors $\mathbf{w}(k)$, i.e.,

$$W_t := [\mathbf{w}(0) \quad \dots \quad \mathbf{w}(N_t - 1)].$$

Thereby, each disturbance vector $\mathbf{w}(k)$, $k = 0, \dots, N_t - 1$, should be sampled from a uniform probability distribution defined on the polytope $\{\mathbf{w} \in \mathbb{R}^{n_x} \mid H_w \mathbf{w} \leq \mathbf{h}_w\}$. Implement a function `generate_disturbances_cwo` that takes `params_z` as input and outputs W_t .

26. Simulate the closed-loop system (21) in the presence of additive disturbances. Adapt the simulation function developed in Task 7 to implement a function `simulate_uncertain`, which takes as inputs $\mathbf{x}_0$, `ctrl`, W_t , `params_z`, and outputs X_t , U_t , and the N_t -dimensional struct `ctrl_info` just as in Task 7, but this time for the uncertain system model (21).

The following tasks lead you through the design of a tube-based robust MPC controller⁴ of the form

$$\min_{Z, V} l_f(\mathbf{z}_N) + \sum_{i=0}^{N-1} \mathbf{z}_i^\top Q \mathbf{z}_i + \mathbf{v}_i^\top R \mathbf{v}_i \quad (23a)$$

$$\text{s.t. } \mathbf{x}(k) \in \mathbf{z}_0 \oplus \mathcal{E} \quad (23b)$$

$$\mathbf{z}_{i+1} = A\mathbf{z}_i + B\mathbf{v}_i, \quad i = 0, \dots, N-1 \quad (23c)$$

$$\mathbf{z}_i \in \mathcal{X}_z \ominus \mathcal{E}, \quad i = 0, \dots, N \quad (23d)$$

$$\mathbf{v}_i \in \mathcal{U}_z \ominus K_{\text{tube}} \mathcal{E}, \quad i = 0, \dots, N-1 \quad (23e)$$

$$\mathbf{z}_N \in \mathcal{X}_N, \quad (23f)$$

where $Z := \{\mathbf{z}_0, \dots, \mathbf{z}_N\}$ and $V := \{\mathbf{v}_0, \dots, \mathbf{v}_{N-1}\}$ define a sequence of nominal states and inputs, $\mathcal{E}$ is a robust positively invariant set⁵, K_{tube} a stabilizing linear feedback controller for system (21), and $\mathcal{X}_z = \{\mathbf{x}_z \in \mathbb{R}^{n_x} \mid H_x \mathbf{x}_z \leq \mathbf{h}_x\}$ and $\mathcal{U}_z = \{\mathbf{u}_z \in \mathbb{R}^{n_u} \mid H_u \mathbf{u}_z \leq \mathbf{h}_u\}$ denote the polytopic state and input constraints. At each time step k , the following control input is applied to the system:

$$\mathbf{u}_z(k) = \kappa_{\text{tube}}(\mathbf{x}_z(k)) = \mathbf{v}_0^* + K_{\text{tube}}(\mathbf{x}_z(k) - \mathbf{z}_0^*). \quad (24)$$

27. In this task, design a stabilizing linear feedback controller $\mathbf{u}_z(k) = K_{\text{tube}} \mathbf{x}_z(k)$ with $K_{\text{tube}} \in \mathbb{R}^{n_u \times n_x}$ for system (21) using pole placement. Implement a function `compute_tube_controller` that takes an array of poles $p = [p_1 \quad p_2]$ with $p_1, p_2 \in \mathbb{C}$ as an input, and outputs the feedback matrix of the tube controller, K_{tube} .

Hint: Use the MATLAB function `place`.

28. Compute the polytopic tube

$$\mathcal{E} := \{\mathbf{x}_z \in \mathbb{R}^{n_x} \mid H_{\text{tube}} \mathbf{x}_z \leq \mathbf{h}_{\text{tube}}\}$$

used in the MPC problem (23) as the minimal robust positive invariant (RPI) set for system (21) under the tube controller designed in Task (27). In a function `compute_minRPI`, implement the algorithm given in the MPC lecture slides⁵ to obtain the polytopic tube $\mathcal{E}$ in dependence of the inputs K_{tube} and `params`. The outputs of the function are given as H_{tube} , $\mathbf{h}_{\text{tube}}$, n_{iter} , where n_{iter} is the number of iterations after which the algorithm has converged, i.e., $n_{\text{iter}} := i$, such that $\mathcal{E}_{i+1} = \mathcal{E}_i$.

Hint 1: To check convergence of your algorithm you can use `eq(\mathcal{E}_{i+1}, \mathcal{E}_i)` after reducing the polytopes to a minimal representation, e.g., by using the function `Polyhedron.minHRep()`.

Hint 2: If your algorithm takes long to converge, consider choosing different poles p to design your tube controller K_{tube} .

⁴Lec. 9, Robust MPC, Tube-MPC Problem Formulation

⁵Lec. 9, Robust MPC, Minimum Robust Invariant Set

29. Obtain the tightened constraints as defined in the MPC problem (23). Let H_x , $\mathbf{h}_x$, H_u , and $\mathbf{h}_u$ define the polytopic constraints for the system (21) as given in `params_z`. Implement a function `compute_tightening`, which takes `params_z` as input and adapts the parameters H_x , $\mathbf{h}_x$, H_u , and $\mathbf{h}_u$ to represent the tightened constraints in (23d) and (23e). The output is then given by the modified struct `params_z_tube`.
30. Implement the tube-based robust model predictive controller (23) in the class `MPC_TUBE`. Design the terminal cost $l_f(\mathbf{x})$ as the LQR infinite-horizon cost (13) of the nominal system, i.e., system (21) without disturbances $w(k)$. During initialization of the class, a YALMIP solver object representing the optimization problem (23) needs to be created based on the inputs Q , R , N , the polytopic terminal set described by H_N and $\mathbf{h}_N$, the polytopic tube $\mathcal{E}$ described by H_{tube} , $\mathbf{h}_{\text{tube}}$, the tube controller K_{tube} , as well as the system model with tightened constraints in the struct `params_z_tube`. The `eval` method should solve the optimization problem and obtain the control input according to (24).
31. Let $Q = \text{diag}(q_z^*, q_{v_z}^*)$, $R = 1$, $N = 50$, and choose $p = [0.05 \quad 0.1]$ to design the tube controller K_{tube} using the function designed in Task 27. Design the tube $\mathcal{E}$ as outlined in Task 28 and obtain tightened constraints as outlined in Task 29. Using the function `lqr_maxPI` with an appropriate choice of inputs, design the terminal set $\mathcal{X}_N$ such that the resulting controller is recursively feasible and robustly stable. Write the design of all ingredients and the setup of the resulting `MPC_TUBE` controller in a script called `MPC_TUBE_script_cwo.m`. Provide the script and save the following variables in the MAT-file `MPC_TUBE_params_cwo.mat`: $p := p$, $K_{\text{tube}} := K_{\text{tube}}$, $H_{\text{tube}} := H_{\text{tube}}$, $\mathbf{h}_{\text{tube}} := \mathbf{h}_{\text{tube}}$, $H_N := H_N$, $\mathbf{h}_N := \mathbf{h}_N$, and `params_z_tube`. Use the naming convention specified in Table 13.
32. [Simulation]: Sample different disturbance sequences W_t and simulate the MPC-TUBE controller obtained in Task 31 using your function `simulate_uncertain` for initial condition $\mathbf{x}_{z,0}^A$. Compare your MPC-TUBE controller against applying the MPC-TS controller implemented in Task 20 based on the system defined in `params_z` with $N = 50$. Plot the closed-loop trajectories using the function `plot_trajectory_z_cwo`. What can you observe? What can you observe if you apply the maximum disturbance at each time step, i.e., $w(k) = [w_{\max} \quad w_{\max}]^T$ for all time steps k ?

Task	Function	Inputs	Outputs
25	<code>generate_disturbances_cwo</code>	<code>params_z</code>	W_t
26	<code>simulate_uncertain</code>	$\mathbf{x}_0$, <code>ctrl</code> , W_t , <code>params_z</code>	X_t , U_t , <code>ctrl_info</code>
27	<code>compute_tube_controller</code>	p , <code>params_z</code>	K_{tube}
28	<code>compute_minRPI</code>	K_{tube} , <code>params_z</code>	H_{tube} , $\mathbf{h}_{\text{tube}}$, n_{iter}
29	<code>compute_tightening</code>	K_{tube} , H_{tube} , $\mathbf{h}_{\text{tube}}$, <code>params_z</code>	<code>params_z_tube</code>
30	<code>MPC_TUBE</code>	Q , R , N , H_N , $\mathbf{h}_N$, H_{tube} , $\mathbf{h}_{\text{tube}}$, K_{tube} , <code>params_z_tube</code>	<code>ctrl</code> (<code>MPC_TUBE</code> object)
30	<code>MPC_TUBE/eval</code>	$\mathbf{x}$	$\mathbf{u}$, <code>ctrl_info</code>
31	<code>MPC_TUBE_script_cwo</code>	-	<code>MPC_TUBE_params_cwo.mat</code>

Table 12: Deliverable summary "Robust MPC"

Name	Variable
p	ρ
K_tube	K_{tube}
H_tube	H_{tube}
h_tube	$\boldsymbol{h}_{\text{tube}}$
H_N	H_N
h_N	$\boldsymbol{h}_N$
params_z_tube	Output of compute_tightening

Table 13: Variable naming convention for MPC_TUBE_params_cwo.mat